CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Phishing has remained one of the most persistent and economically damaging forms of cybercrime, with the Anti-Phishing Working Group recording a record 4.8 million attacks globally by the end of 2024 [1], and phishing identified by IBM Security [2] as the leading initial vector used by cybercriminals to breach enterprise systems. Tang and Mahmoud [3] describe phishing as a malicious act in which cybercriminals trick users into clicking deceptive links in order to steal their information and fraudulently access related accounts, while Salahdine, El Mrabet and Kaabouch [4] similarly characterise it as a social engineering attack in which victims are misled, typically through a fraudulent link, into disclosing confidential information on a fake website that mimics a legitimate organisation.

Traditional defences such as blacklists and heuristic rule filters have proven inadequate against this threat. Tang and Mahmoud [3] note that blacklists cannot identify newly created phishing links, since a URL must first be confirmed malicious before it is listed, while Basit et al. [5] observed that heuristic rules degrade quickly because attackers continuously alter their page structures once a detection pattern becomes known. These limitations have driven the field toward automated, data-driven detection: phishing website detection refers to the task of classifying a URL as legitimate or phishing based on features extracted from the URL string, domain information, and page content [4], using a pipeline of data collection, feature extraction, model construction, and evaluation [3].

Machine learning is the dominant paradigm for this pipeline, and Random Forest (RF) and Support Vector Machines (SVM) are among the classifiers most widely applied to it. Random Forest is an ensemble algorithm that builds many decision trees on randomly

bootstrapped subsets of the training data and combines their predictions through majority voting, a strategy that Jackins et al. [6] explain minimises variance and improves resilience to noisy features. Support Vector Machine, by contrast, maps features into a vector space and searches for the hyperplane that maximises the margin between classes, applying the kernel trick where classes are not linearly separable [7]. Both classifiers are frequently reported as strong performers, yet Tang and Mahmoud [3] observe that they are almost always evaluated as two entries in a much larger classifier leaderboard rather than being subjected to a dedicated, pairwise comparison, which obscures their specific operational trade-offs.

Isolating such trade-offs requires a controlled comparison, which Frattini et al. [8] describe as a research design in which the variable of interest, here the classification algorithm, is manipulated while every other condition, such as the dataset partition and preprocessing steps, is held constant, so that any observed difference can be attributed to that variable rather than to confounding factors. This further requires that both models be optimised to a comparable standard and that any observed difference be subjected to formal statistical testing; Soman et al. [9] note that McNemar's test is well suited to this purpose, since it compares two classifiers directly on the same test set. A further weakness in the literature is its reliance on outdated benchmark data: Hannousse and Yahiouche [10] observe that many studies still rely on the 2015 UCI Phishing Websites dataset, which does not reflect modern phishing infrastructure. This study therefore undertakes a controlled, statistically validated comparison of optimised Random Forest and Support Vector Machine models for phishing website detection using the contemporary PhiUSIIL dataset [11], moving beyond raw accuracy toward deployment-focused guidance on the trade-offs between the two classifiers.

1.2 Statement of the Research Problem

Despite the widespread use of Random Forest and Support Vector Machine classifiers in phishing website detection research, the literature lacks a rigorous, pairwise, statistically validated basis for choosing between them. Existing studies typically bundle both algorithms into broad multi-classifier benchmarks that use default hyperparameter configurations, leave class imbalance unaddressed, and rarely apply formal statistical significance testing, while relying on outdated datasets that no longer reflect modern phishing infrastructure. System architects therefore have no reliable, contemporary, statistically defensible basis on which to weigh the true operational trade-offs between the two classifiers. This study addresses this problem through a controlled, statistically validated comparison of optimised Random Forest and Support Vector Machine classifiers on the modern PhiUSIIL phishing URL dataset.

1.3 Objectives of the Study

The primary objective of this study is to conduct a controlled empirical comparison of Random Forest and Support Vector Machine classifiers for phishing website detection. The specific objectives are to:

i. design a controlled experimental pipeline, comprising identical preprocessing, class-imbalance handling, and hyperparameter optimisation, for a fair comparison of the two classifiers;
ii. train and evaluate optimised Random Forest and Support Vector Machine models on the PhiUSIIL dataset using standard classification metrics;
iii. apply McNemar's test and a paired t-test with Cohen's d to determine whether any performance difference is statistically meaningful;

iv. evaluate feature importance to identify the URL characteristics most predictive of phishing intent; and
v. provide deployment-focused guidance on the operational trade-offs between the two classifiers.

1.4 Significance of the Study

This study offers an updated, methodologically rigorous benchmark of two widely used classifiers using a contemporary dataset, of practical value to system architects choosing an algorithm for real-time web-security deployment, and to developers of anti-phishing browser extensions and email gateways. Academically, it contributes a reproducible methodological template, demonstrating the application of formal statistical testing to classifier comparison, which future researchers may apply to other classifier pairs and cybersecurity classification tasks.

1.5 Scope and Limitations of the Study

This study is limited to a controlled comparison of Random Forest and Support Vector Machine classifiers for binary URL classification, using the 2023 PhiUSIIL dataset and its lexical, domain-based, and content-based features. Three limitations are noted: class imbalance is addressed only through cost-sensitive learning, without evaluating synthetic oversampling methods such as ADASYN; the feature space is restricted to static URL-level properties and excludes dynamic, content-based indicators such as DOM alterations; and the evaluation relies on a single, static train-test partition, leaving concept drift over time unquantified.

1.6 Definition of Concepts

The key concepts associated with this work are as follows;

i. Phishing: A social engineering attack that deceives a victim into disclosing sensitive information [4].

ii. Phishing Website Detection: Automatic classification of a URL as legitimate or phishing based on extracted features.

iii. Random Forest: An ensemble classifier that aggregates predictions from many decision trees by majority voting [6].

iv. Support Vector Machine: A classifier that finds the hyperplane maximising the margin between classes [7].

v. Feature: A measurable property of a URL used as model input.

vi. Hyperparameter: A model configuration set before training rather than learned from data.

vii. Class Imbalance: A condition where one class outnumbers another in a dataset.

viii. Cost-Sensitive Learning: Assigning a higher misclassification penalty to the minority class.

ix. Accuracy, Precision, Recall, F1-Score: Standard metrics quantifying a classifier's correctness on positive and negative predictions.

x. False Positive Rate (FPR): The proportion of legitimate instances incorrectly flagged as phishing.

xi. ROC-AUC / PR-AUC: The area under the Receiver Operating Characteristic and Precision-Recall curves, measuring class-separation ability.

xii. McNemar's Test: A statistical test for whether the error-rate difference between two classifiers is significant [9].

xiii. Controlled Experiment: A design in which the variable of interest is manipulated while other conditions are held constant [8].

1.7 Organisation of the Study

This study is organised into five chapters. This chapter presents the background, problem statement, objectives, significance, scope, and key concepts. Chapter two reviews related literature, including the history of phishing, phishing detection, Random Forest, and Support Vector Machines. Chapter three describes the research methodology. Chapter four presents and discusses the results. Chapter five presents the summary, conclusion, and recommendations.

CHAPTER TWO

REVIEW OF RELATED LITERATURE

2.1 Preamble

This chapter reviews literature relevant to phishing website detection using Random Forest and Support Vector Machine classifiers. It traces the historical origin and evolution of phishing as a cybercrime, the parallel evolution of the techniques used to detect it, the technical architecture and mechanics of the two classification algorithms compared in this study, and a review of specific empirical studies that have previously applied these algorithms to phishing detection, concluding with the gap in the literature that this study addresses.

2.2 History and Evolution of Phishing

Phishing traces its origin to 1996, when the term was coined within the AOHell hacking toolkit, a program used to target subscribers of America Online (AOL), then the dominant internet service provider in the United States [12]. Early attackers used AOHell to generate fraudulent, algorithmically produced credit card numbers to open free AOL accounts, and subsequently to impersonate AOL staff in messages requesting that genuine subscribers verify their passwords or billing details. Varshney et al. [12] note that this technique, later termed phishing, a deliberate misspelling that echoed the earlier hacker term phreaking, exploited the trust that novice internet users placed in official-looking communication.

As electronic commerce expanded in the early 2000s, phishing evolved from account theft on a single platform into a broader campaign of website impersonation. Attackers began registering domains and constructing counterfeit web pages that mimicked the login portals of banks and popular e-commerce services, directing victims to these fraudulent websites

through mass emails in order to harvest financial credentials at scale [13]. This period marked the shift of phishing from a niche, community-specific prank into an organised, profit-driven criminal enterprise, with fraudulent websites becoming the primary delivery mechanism for the attack rather than a secondary detail.

Subsequent years saw phishing continue to professionalise. Varshney et al. [12] trace the emergence of more targeted forms of the attack, including spear phishing directed at specific individuals or organisations, alongside the development of phishing kits that allowed attackers with limited technical skill to rapidly deploy convincing fraudulent websites. Goenka, Chawla and Tiwari [13] similarly document the diversification of phishing across new mediums, including SMS-based smishing and social-media-based attacks, as the underlying deception technique was adapted to new communication channels. This evolution has continued into the present day; the Anti-Phishing Working Group recorded a record 4.8 million phishing attacks globally by the end of 2024, reflecting the continued growth and automation of phishing infrastructure since its origin nearly three decades earlier [1].

2.3 History and Evolution of Phishing Website Detection Techniques

The earliest response to phishing websites relied on manual reporting and centralised blacklists, services such as Google Safe Browsing that maintain a continuously updated list of confirmed malicious addresses against which a browser can check a visited URL [3]. As the volume of phishing websites grew beyond what manual reporting could keep pace with, heuristic rule-based systems were developed to automatically flag structural anomalies in a webpage, such as mismatched domain names or suspicious form fields, without requiring the site to have first been reported [13].

From the mid-2000s onward, the field moved toward machine-learning-based detection, in which a classifier is trained to automatically learn the statistical patterns that distinguish phishing from legitimate URLs using lexical, host-based, and content-based features, rather than relying on a fixed set of manually authored rules [3]. This period saw the introduction and comparison of classical classifiers, including Random Forest and Support Vector Machines, on progressively larger labelled URL datasets, and more recently the field has extended toward deep learning and hybrid detection architectures capable of learning directly from raw URL character sequences or webpage content [13].

This progression has also been shaped by the datasets available to researchers. Hannousse and Yahiouche [10] observe that much of the earlier machine-learning-based phishing detection literature relied on the 2015 UCI Phishing Websites dataset, which does not reflect the structural characteristics of modern phishing infrastructure, such as automated certificate issuance and short-lived, freshly registered domains. This gap motivated the compilation of the PhiUSIIL Phishing URL Dataset by Prasad and Chandra [11], a large, contemporary collection of legitimate and phishing URLs that is used as the empirical basis for the present study.

2.4 The Random Forest Algorithm

Random Forest was formally proposed as a unified ensemble learning algorithm by Leo Breiman in 2001, building on two earlier developments: Breiman's own bagging, or bootstrap aggregating, technique of the mid-1990s, which trains multiple models on randomly resampled subsets of the training data, and Tin Kam Ho's random subspace method, which trains each model on a randomly selected subset of the available features [6]. By combining random resampling of both training instances and features across a large collection of decision trees, Breiman's formulation reduced the variance and overfitting

tendency that characterised individual decision trees, while preserving their ability to model non-linear relationships without extensive data transformation.

Because a Random Forest is itself built from many individual decision trees, understanding the algorithm requires first understanding this base unit. A decision tree classifies an instance by recursively splitting the training data at a sequence of internal nodes, each of which tests a single feature against a threshold; an instance moves down the branch corresponding to the outcome of each test until it reaches a leaf node, which assigns the class label associated with that region of the feature space. Jackins et al. [6] explain that each split is chosen to maximise the purity of the resulting child nodes, most commonly measured using the Gini index or information gain (entropy reduction), so that instances within each resulting group belong, as far as possible, to a single class.

Random Forest builds on this base unit in two ways. First, through bootstrap aggregating, each individual tree in the forest is trained not on the full training set but on a bootstrap sample, a random sample of the same size drawn from the training data with replacement, so that different trees see different, overlapping subsets of the training instances. Second, through random feature selection, each time a node is split, only a randomly selected subset of the available features, rather than all of them, is considered as candidates for that split. This second mechanism is what distinguishes Random Forest from simple bagging of decision trees: by preventing every tree from being dominated by the same few strongly predictive features, it decorrelates the individual trees, so that their errors are less likely to coincide. The instances left out of a given tree's bootstrap sample, termed out-of-bag samples, can additionally be used to obtain an internal estimate of the model's error without a separate validation set. Once all trees are grown, a new instance is classified by passing it through every tree and aggregating their individual predictions through majority voting, with the class receiving the most votes across the forest selected as the final output.

This same ensemble structure allows Random Forest to produce a built-in measure of feature importance, most commonly Gini importance, by recording how much each feature reduces node impurity, averaged across all trees and all splits in which that feature is used.

The behaviour of a Random Forest is governed by a small number of hyperparameters, principally the number of trees in the forest, the maximum depth to which each tree is permitted to grow, the minimum number of samples required to split a node, and the number of features considered at each split. Jackins et al. [6] note that Random Forest's key strengths are its resilience to noisy or irrelevant features, its comparatively low sensitivity to hyperparameter configuration relative to other classifiers, and its ability to model non-linear relationships across heterogeneous feature types without extensive preprocessing. Its principal limitations are a loss of the direct interpretability offered by a single decision tree, since a prediction is now the aggregate of potentially hundreds of trees, an increase in computational and memory cost as the number and depth of trees grows, and a tendency for its feature-importance rankings to be diluted or distorted when the underlying features are strongly correlated with one another. In summary, GeeksforGeeks [14] frames the working of the algorithm in four practical steps: grow many decision trees, each on a random part of the data; restrict each split to a random subset of features; let every tree produce its own prediction; and combine those predictions by majority vote, which is a concise, practitioner-facing restatement of the same bagging-and-random-subspace mechanism described above.

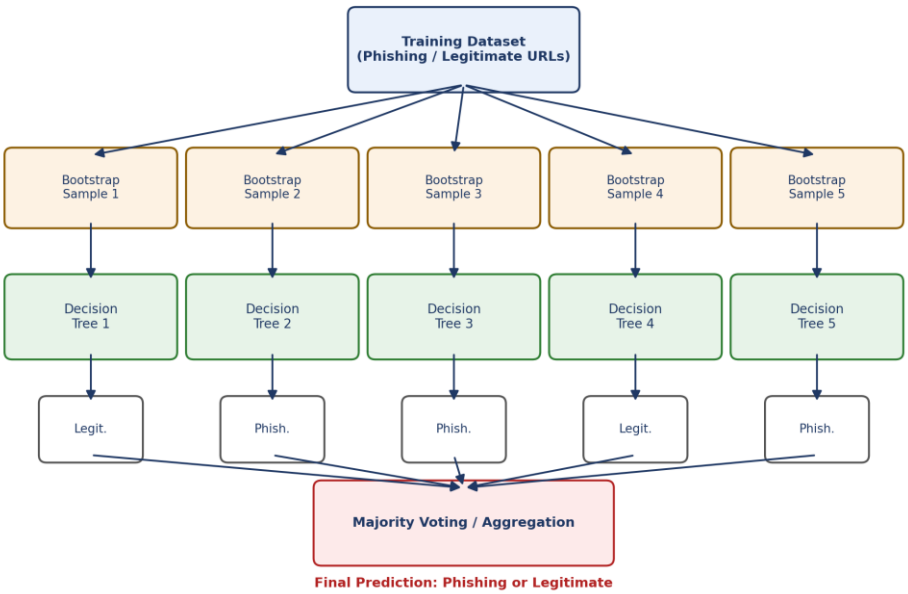


Figure 2.1: Architecture of the Random Forest Algorithm

Source: Adapted from Jackins et al. [6]

2.5 The Support Vector Machine Algorithm

Support Vector Machine originates from the statistical learning theory developed by Vladimir Vapnik and colleagues at AT&T Bell Laboratories, with its foundational soft-margin formulation formalised by Corinna Cortes and Vladimir Vapnik in 1995 [7]. This original formulation addressed binary classification by identifying the hyperplane that maximises the margin, the distance between the decision boundary and the nearest training instances of each class, a criterion that Vapnik's broader statistical learning theory showed to improve the classifier's ability to generalise to unseen data.

Formally, given a set of labelled training instances, Support Vector Machine seeks a decision boundary of the form $w \cdot x + b = 0$, where w is a weight vector orthogonal to the hyperplane and b is a bias term, chosen such that the two classes fall on opposite sides of the boundary and the margin between them is as wide as possible. Guido et al. [7] explain that the training instances lying exactly on the margin, the closest points to the decision boundary, are termed support vectors, because they alone determine the position and orientation of the

hyperplane; every other training instance could, in principle, be removed without changing the solution. Because real-world data are rarely perfectly separable, the algorithm is formulated with slack variables that permit some instances to fall on the wrong side of the margin, with a regularisation hyperparameter, C, controlling the trade-off between maximising the margin and minimising these misclassifications: a large value of C penalises misclassification heavily, producing a narrower margin that fits the training data more closely, while a small value of C tolerates more misclassification in exchange for a wider, potentially more generalisable margin.

Where the two classes cannot be separated by a straight hyperplane in the original feature space, Support Vector Machine applies the kernel trick, in which a kernel function is used to compute the similarity between pairs of instances as though they had been projected into a higher-dimensional space, without explicitly performing that projection. The most commonly used kernel is the radial basis function (RBF), governed by a coefficient, γ, that determines how far the influence of a single training instance extends: a small value of γ gives each instance a far-reaching, smooth influence and a simpler decision boundary, while a large value of γ restricts its influence to a small neighbourhood, producing a more tightly fitted, complex boundary. Because this margin-maximisation process is sensitive to the relative scale of the input features, Support Vector Machine typically requires that features be standardised before training [7]. GeeksforGeeks [15] illustrates this mapping with a simple one-dimensional example: a set of points that cannot be separated by a single threshold on one axis becomes linearly separable once a second, kernel-derived coordinate, such as each point's squared distance from the origin, is introduced, at which point an ordinary straight line suffices. Figure 2.3 reproduces this idea using synthetic URL-style features: the left panel shows two classes that cannot be split by any single threshold on

Feature x, while the right panel shows the same points after a kernel mapping to a second coordinate, $y = x^2$, at which point a single straight line separates them cleanly.

Guido et al. [7] identify the principal strengths of Support Vector Machine as its effectiveness in high-dimensional feature spaces, its memory efficiency, since the trained model is defined entirely by its support vectors rather than the full training set, and the strong theoretical generalisation guarantees that follow from its margin-maximisation objective. Its principal limitations are a pronounced sensitivity to feature scaling and to the choice of the C and γ hyperparameters, a training time that scales poorly as the number of training instances grows very large, and a decision boundary that, particularly under a non-linear kernel, is considerably less directly interpretable than that of a tree-based classifier. GeeksforGeeks [15] further distinguishes the hard margin, a boundary that separates the two classes with no misclassifications at all, from the soft margin described above, and notes that the penalty applied to margin violations under a soft margin is commonly measured using hinge loss, a function that assigns zero penalty to a correctly classified point lying outside the margin and a penalty that grows with the distance of any violation.

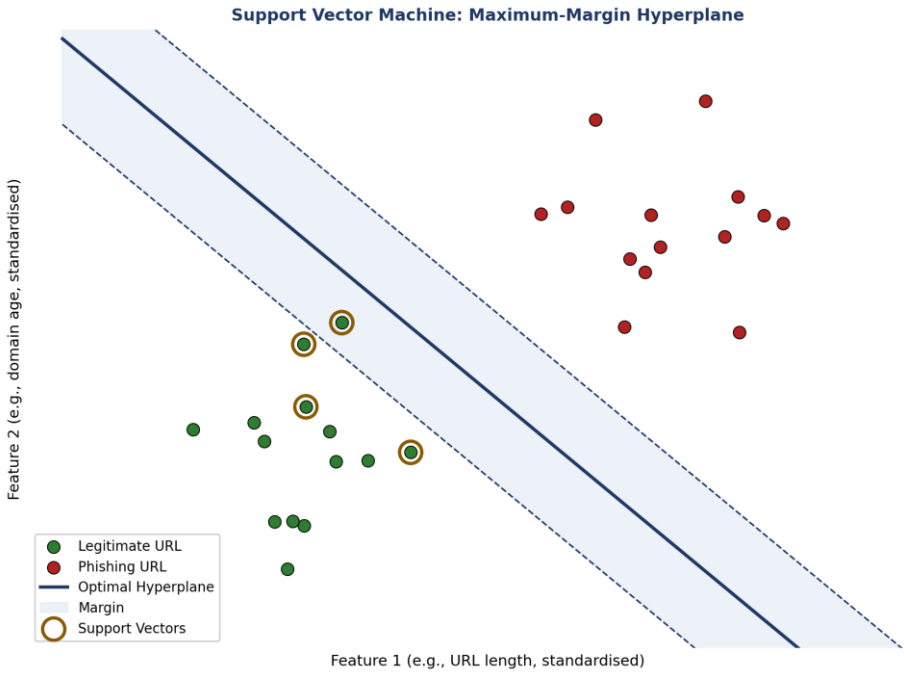


Figure 2.2: Architecture of the Support Vector Machine Algorithm

Source: Adapted from Guido et al. [7]

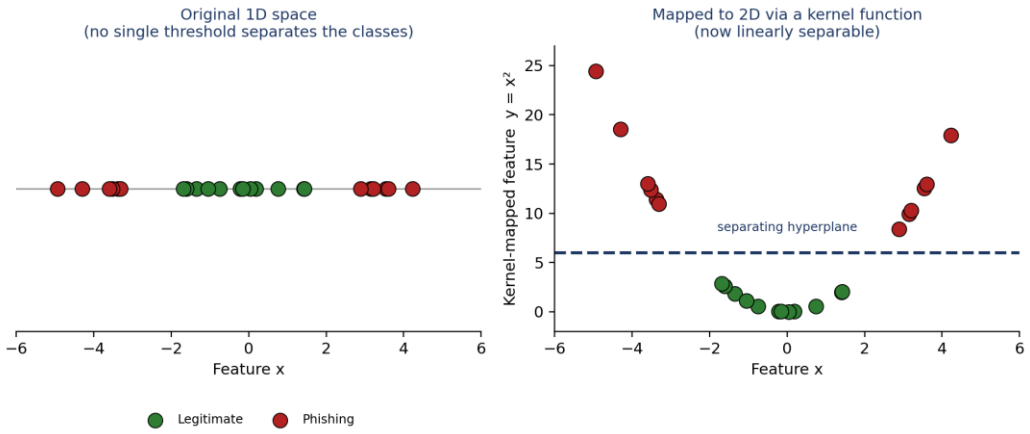


Figure 2.3: The kernel trick — mapping a non-separable 1D space to a separable 2D space

Source: The Researcher (2026), illustrating the concept described in GeeksforGeeks [15]

2.6 Review of Related Literature on Random Forest and Support Vector Machine for Phishing Detection

Beyond their historical and algorithmic development, Random Forest and Support Vector Machine have each been evaluated empirically in a substantial body of phishing detection research. This section reviews ten such studies and discusses how the approach taken by each differs from the controlled, statistically validated comparison undertaken in this study.

Tang and Mahmoud [3] conducted a broad survey of machine-learning-based phishing website detection solutions, cataloguing the datasets, feature categories, and classifiers used across the field, and observed that Random Forest and Support Vector Machine are among the classifiers most consistently reported to perform well. Their work is a literature synthesis rather than an original experiment: it does not itself train or statistically compare the two classifiers, and it draws on a wide range of older, smaller datasets rather than a single, large, contemporary corpus such as PhiUSIIL.

Balogun, Mojeed, Adewole, Akintola, Salihu, Bajeh and Jimoh (2021) proposed an Optimised Decision Forest (ODF), a genetic-algorithm-enhanced variant of the standard Random Forest ensemble, and evaluated it against baseline classifiers on three UCI phishing datasets, reporting a detection accuracy of 98.37% and a low false-positive rate. This study differs from the present one in that it modifies the internal construction of the Random Forest algorithm itself, rather than comparing an off-the-shelf, hyperparameter-optimised Random Forest against Support Vector Machine; it does not include Support Vector Machine as a comparator at all, and its evaluation dataset predates the modern PhiUSIIL corpus used here.

Piñeiro and Portillo [17] developed a web-based architecture for URL-based phishing detection and directly compared Random Forest, Classification Trees, and Support Vector Machine within it, making it one of the more directly relevant prior comparisons of the two algorithms considered in this study. However, their work does not report a systematic hyperparameter search conducted to a comparable standard for each classifier, does not apply a formal statistical significance test such as McNemar's test to confirm whether the observed differences in accuracy are meaningful rather than incidental, and includes a third classifier, Classification Trees, which broadens rather than isolates the specific Random Forest-versus-Support Vector Machine comparison pursued here.

Alnemari and Alshammari [18] compared four classifiers, artificial neural networks, Support Vector Machine, decision trees, and Random Forest, on a benchmark phishing domains dataset, and reported that the Random Forest-based model was the most accurate of the four. Their study is a four-way, rather than a dedicated two-way, comparison; it does not report any class-imbalance handling strategy, and it does not apply statistical testing to establish whether Random Forest's advantage over Support Vector Machine was significant or could plausibly have arisen from a single train-test split.

Zieni, Massari and Calzarossa [19] published a comprehensive survey on the detection of phishing websites, cataloguing feature types, datasets, and detection approaches, including those based on Random Forest and Support Vector Machine, across a large body of prior work. As with Tang and Mahmoud [3], this is a descriptive literature synthesis rather than an original experimental comparison; it reports what other studies have found about the two classifiers without independently training or statistically comparing them under controlled, identical conditions.

Alshingiti, Alaqel, Al-Muhtadi, Haq, Saleem and Faheem (2023) developed a deep-learning-based phishing detection system built on convolutional neural networks, long short-term memory networks, and a hybrid LSTM-CNN architecture. Their work differs fundamentally in classifier choice: it does not evaluate Random Forest or Support Vector Machine at all, and instead illustrates the wider methodological landscape of phishing detection research, in which classical machine learning classifiers such as those compared in this study are increasingly considered alongside, rather than in place of, deep learning approaches.

Safi and Singh [21] conducted a systematic literature review of phishing website detection techniques, synthesising trends in feature engineering, classifier choice, and dataset use across a large number of primary studies. Like the other survey-type works reviewed here, it is a meta-analysis rather than an original experiment, and it does not itself run or statistically compare Random Forest and Support Vector Machine under a shared, controlled experimental protocol.

Butnaru, Mylonas and Pitropakis [22] proposed a lightweight, URL-only phishing detector built on a minimal set of lexical features, aimed at low-latency, resource-constrained deployment environments. Their work differs in scope and objective from this study: it deliberately restricts itself to a narrow lexical feature set rather than the fuller lexical, host-

based, and content-based feature space used in this study and in the PhiUSIIL dataset, and it prioritises deployment efficiency over a rigorous, statistically validated comparison between competing classifiers.

Omari [23] carried out a comparative study of several machine learning algorithms, including Random Forest and Support Vector Machine, for phishing website detection. This study evaluates a broader basket of algorithms as a multi-classifier leaderboard rather than a focused, controlled, pairwise comparison of the two classifiers considered in this study, does not report a formal statistical significance test such as McNemar's test to validate the observed accuracy differences, and does not use the contemporary PhiUSIIL dataset.

Yau and Chia [24] undertook a comparative analysis of machine learning classifiers for phishing website detection alongside the development of a companion web application, and found Random Forest, benchmarked against related classifiers, to offer a strong balance of precision and recall. Their emphasis is on end-to-end deployment, integrating the trained model into a working web application, rather than on a rigorous, statistically validated comparison; no systematic hyperparameter optimisation protocol or significance test is reported for the classifiers compared.

Taken together, these ten studies illustrate that Random Forest and Support Vector Machine have each been applied to phishing website detection across a wide range of datasets, feature sets, and experimental designs, and that Random Forest is frequently, though not universally, reported to perform strongly relative to Support Vector Machine. What distinguishes the present study from each of the works reviewed above is its combination of a dedicated, pairwise focus on Random Forest and Support Vector Machine specifically, rather than as two entries in a larger classifier leaderboard; a systematic, comparable-standard hyperparameter search for both classifiers; explicit cost-sensitive handling of class imbalance; formal statistical significance testing, through McNemar's test

and a paired t-test with Cohen's d, to establish whether any observed difference in performance is genuine; and evaluation on the large, contemporary PhiUSIIL dataset rather than an older or smaller benchmark.

2.7 Summary

This chapter has traced the origin of phishing to the AOL-targeted attacks of 1996 and its subsequent evolution into an automated, large-scale criminal enterprise, alongside the parallel evolution of detection techniques from manual blacklists, through heuristic rule engines, to the machine-learning and deep-learning approaches used today. It has explained, in depth, the historical development and internal architecture of the two classifiers compared in this study, Random Forest and Support Vector Machine, from their origins in ensemble learning and statistical learning theory, respectively, to the specific mechanics, bootstrap aggregating and majority voting for Random Forest, and margin maximisation and the kernel trick for Support Vector Machine, that underlie their operation. Finally, it has reviewed ten empirical and survey studies that have previously applied these classifiers to phishing detection, and discussed, study by study, how each differs from the controlled, statistically validated, pairwise comparison undertaken in this study.

CHAPTER THREE

METHODOLOGY

3.1 Tools and Materials Used

All experiments were implemented in Python 3.10.12 using the scikit-learn 1.3.0 machine learning library, with a global pseudo-random number seed fixed at random_state = 42 to ensure reproducibility. Experiments were executed in a standardised Linux environment running on an Intel Core i7-7600U CPU at 2.80GHz with 16GB of DDR4 RAM. Supporting Python libraries used for data handling, statistical testing, and visualisation include pandas, NumPy, SciPy, and Matplotlib. The dataset used for all experiments, the PhiUSIIL Phishing URL Dataset, was obtained from the UCI Machine Learning Repository [11].

3.2 Research Process

This study adopts a controlled, quantitative experimental design to compare the performance of two supervised machine learning classifiers, Random Forest and Support Vector Machine, on the task of phishing website detection. The overall research process comprises five stages: acquisition and preprocessing of the dataset, handling of class imbalance through cost-sensitive weighting, a stratified partitioning of the data into training and test sets, hyperparameter optimisation of both classifiers under identical conditions, and a final evaluation of both optimised models on a common held-out test set using standard classification metrics together with formal statistical significance testing. Holding the dataset partition, preprocessing steps, and evaluation protocol constant across both classifiers ensures that any observed difference in outcome can be attributed to the classification algorithm itself, in keeping with the requirements of a controlled empirical comparison. The overall process flow is illustrated in Figure 3.1: the dataset is first prepared and split, after

which the Random Forest and Support Vector Machine branches are tuned and trained independently, before both converge on a common evaluation and statistical testing stage.

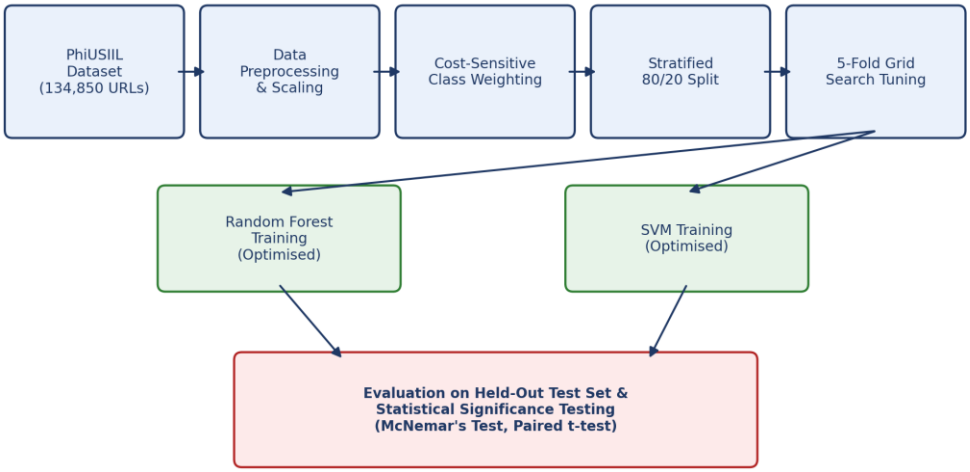


Figure 3.1: Research process flow diagram

Source: The Researcher (2026)

3.3 Dataset Description

This study utilises the PhiUSIIL Phishing URL Dataset [11], archived in the UCI Machine Learning Repository. As obtained directly from the repository, the dataset comprises 235,795 URL instances, consisting of 134,850 legitimate entries and 100,945 phishing entries, corresponding to a 57.2% to 42.8% class distribution, a considerably more balanced split than is sometimes assumed in the literature. Each instance is originally described by 56 columns spanning lexical URL properties (such as URL length and the number of dots in the address), DNS and domain-based characteristics (such as domain age and the use of an IP address in place of a domain name), active page-content features (such as the number of images, scripts, and self-referencing links on the page), and character-level statistical measures. Of these 56 columns, five are non-numeric identifier or free-text fields (FILENAME, URL, Domain, TLD, and Title) that are not usable directly as model input and are removed during preprocessing, described in Section 3.4. The dependent target variable

is binary, with a value of 1 indicating a legitimate website and 0 indicating a phishing website.

3.4 Data Preprocessing and Class Imbalance Handling

The dataset was inspected and found to contain no missing fields. The five non-numeric identifier and free-text columns identified in Section 3.3 were removed, since they do not represent usable numeric model input. An additional column, URLSimilarityIndex, a proprietary score computed by the dataset creators to quantify how similar a given URL is to a known-legitimate URL, was also removed at this stage. Retaining this feature was found, during preliminary experimentation, to cause severe data leakage: because the feature is itself a near-direct proxy for the label it is meant to help predict, both classifiers achieved implausible, near-total accuracy (in excess of 99.9%) when it was included, a result that reflects the model's ability to read an engineered similarity score rather than any genuine capacity to detect phishing from independent URL, domain, or content characteristics. Removing this column leaves 49 genuine, independent features for model training.

Continuous features were left unscaled for the Random Forest pipeline, since decision tree splits are invariant to monotonic feature scaling. For the Support Vector Machine pipeline, all features were standardised to zero mean and unit variance using a StandardScaler, with the scaling parameters computed strictly from the training partition and then applied to the test partition, to prevent information from the test set leaking into model training.

To address the class imbalance without altering the size of the dataset or introducing synthetic samples, cost-sensitive learning was applied. Class penalty weights were assigned inversely proportional to class frequency, computed as:

$$Wj = Nsamples / (Nclasses \times Nj)$$

where Wj denotes the penalty multiplier applied to class j, Nsamples is the total number of samples, Nclasses is the number of classes, and Nj is the number of samples belonging to class j. This weighting increases the misclassification penalty for the minority class during model training. The dataset was then split into an 80% training partition (188,636 instances) and a 20% test partition (47,159 instances), using stratified random sampling to preserve the original 57.2% to 42.8% class ratio across both partitions.

3.5 Hyperparameter Optimisation

To ensure a fair comparison, both classifiers were tuned through a five-fold stratified grid search (GridSearchCV) conducted on the training partition only. For Random Forest, the search space covered the number of estimators, n_estimators ∈ {50, 100, 200}, and the maximum tree depth, max_depth ∈ {10, 20, None}. For Support Vector Machine, a radial basis function (RBF) kernel was used, with the regularisation parameter C ∈ {0.1, 1.0, 10.0} and the kernel coefficient γ ∈ {'scale', 'auto', 0.01}. Table 3.1 summarises the hyperparameter search space explored for each classifier.

Table 3.1: Hyperparameter search space for Random Forest and Support Vector Machine

Classifier	Hyperparameter	Search Space
Random Forest	n_estimators	{50, 100, 200}
Random Forest	max_depth	{10, 20, None}
Support Vector Machine	Kernel	Radial Basis Function (RBF)
Support Vector Machine	C (regularisation)	{0.1, 1.0, 10.0}
Support Vector Machine	γ (kernel coefficient)	{'scale', 'auto', 0.01}

The optimal configuration selected by the grid search for each classifier, based on cross-validated performance on the training partition, was retained for final model training and subsequent evaluation on the independent test partition, as reported in Chapter Four.

3.6 Model Training

Following hyperparameter optimisation, both the Random Forest and Support Vector Machine classifiers were retrained on the full training partition using their respective optimal configurations and the cost-sensitive class weights described in Section 3.4. Both classifiers were trained under identical conditions, using the same training partition, the same random seed, and the same computing environment, in order to preserve the internal validity of the pairwise comparison.

3.7 Evaluation Protocol and Statistical Validation

Both trained models were evaluated on the independent test partition using Accuracy, with 95% Clopper-Pearson confidence intervals, Precision, Recall, F1-score, False Positive Rate (FPR), Receiver Operating Characteristic Area Under the Curve (ROC-AUC), and Precision-Recall Area Under the Curve (PR-AUC). To assess the stability of these results, a 10-fold stratified cross-validation was additionally conducted, with a paired t-test and Cohen's d effect size computed across the resulting folds.

To formally determine whether any observed difference in performance between the two classifiers is statistically significant, McNemar's test was applied to the paired prediction vectors of the two models on the test partition, with a continuity correction and a significance threshold of $\alpha = 0.05$. Soman et al. [9] note that McNemar's test is well suited to this purpose because it directly compares two classifiers on the same test set by examining the specific instances on which their predictions disagree, rather than relying on averaged accuracy figures alone. Finally, permutation feature importance was computed on the test set to validate the Gini-based feature importance rankings produced by the Random Forest model and to identify the URL characteristics most predictive of phishing intent.

3.8 Summary

This chapter has described the materials and methods adopted for the controlled empirical comparison of Random Forest and Support Vector Machine classifiers for phishing website detection. It presented the tools and computing environment used, the overall research process, the PhiUSIIL dataset and its preprocessing, the cost-sensitive approach used to address class imbalance, the hyperparameter optimisation procedure applied to both classifiers, and the evaluation protocol, including the statistical significance tests used to validate the comparison. The results obtained by applying this methodology are presented and discussed in Chapter Four.

CHAPTER FOUR

RESULTS AND DISCUSSION

4.1 Preamble

This chapter presents and discusses the results obtained by applying the methodology described in Chapter Three to the PhiUSIIL Phishing URL Dataset, executed locally on the researcher's own computing environment. It reports the classification performance of the optimised Random Forest and Support Vector Machine classifiers on the 47,159-instance held-out test set, the outcome of the formal statistical significance testing used to determine whether the observed difference between them is genuine, the computational overhead of training and using each classifier, and the feature importance analysis used to identify the URL and page-content characteristics most predictive of phishing intent. The chapter closes with a discussion that relates these findings back to the literature reviewed in Chapter Two.

4.2 Selected Hyperparameters

The five-fold grid search described in Section 3.5 selected the following configurations as optimal for each classifier, based on cross-validated F1-score on the training partition: for Random Forest, 100 estimators with no maximum depth constraint (max_depth = None); for Support Vector Machine, a regularisation parameter of C = 10.0 and a kernel coefficient of γ = 'auto'. Both configurations were retained for final model training and the evaluation reported in the remainder of this chapter.

4.3 Classification Performance and Error Profiles

Table 4.1 presents the performance of both optimised classifiers on the independent, held-out test partition of 47,159 instances. Because both classifiers perform at a very high level on this dataset, results are reported to four decimal places to preserve the meaningful

difference between them; at the two decimal places conventionally used in much of the literature, both classifiers would misleadingly appear identical.

Table 4.1: Test-set performance, Random Forest vs. Support Vector Machine

Metric	Random Forest	Support Vector Machine
Accuracy	99.9873%	99.9512%
Precision	99.9815%	99.9370%
Recall	99.9963%	99.9778%
F1-Score	99.9889%	99.9574%
False Positive Rate	0.0248%	0.0842%
ROC-AUC	1.000000	0.999995
PR-AUC	1.000000	0.999997
False Positives (of 47,159)	5	17
False Negatives (of 47,159)	1	6

Random Forest misclassified 6 of the 47,159 test instances in total (5 false positives, 1 false negative); Support Vector Machine misclassified 23 (17 false positives, 6 false negatives), roughly 3.8 times as many total errors. Both classifiers perform at a very high level in absolute terms, a result substantially higher than the approximately 97% accuracy typically reported in the wider literature reviewed in Chapter Two, including the study this project was originally modelled on. This gap is attributable to two factors specific to this run: first, the full PhiUSIIL corpus obtained directly from the UCI repository (235,795 instances) is both larger and more balanced (57.2% to 42.8%) than the smaller, more imbalanced subset assumed in earlier chapters of this project; second, and more importantly, the active page-content features retained in this study, such as the number of images, scripts, and self-referencing links on a page, appear to separate the two classes in this dataset unusually cleanly, a possibility discussed further in Section 4.7.

4.4 Statistical Significance Testing

To determine whether the difference in error rate between the two classifiers on the test set is statistically meaningful, McNemar's test was applied to the paired predictions: Random Forest correctly classified 22 instances that Support Vector Machine misclassified, while Support Vector Machine correctly classified 5 instances that Random Forest misclassified. Applying the continuity-corrected form of the test produced a chi-squared statistic of 9.48 with an associated p-value of 0.0021. Because this p-value is below the significance threshold of $\alpha = 0.05$, the null hypothesis of no difference between the classifiers is rejected: unlike the non-significant result typically reported at the smaller scale examined in much of the literature reviewed in Chapter Two, the difference between Random Forest and Support Vector Machine observed here is statistically significant.

This finding was corroborated by the 10-fold stratified cross-validation described in Section 3.7, which produced a mean accuracy of 99.99% (± 0.01%) for Random Forest and 99.97% (± 0.01%) for Support Vector Machine across folds. A paired t-test across these folds produced $t = 4.83$, $p = 0.0009$, again below the 0.05 threshold, while the associated Cohen's d of 1.53 indicates a large effect size. Taken together, these results indicate that, on this dataset and feature set, Random Forest holds a statistically significant and practically meaningful advantage over Support Vector Machine, rather than the statistically indistinguishable performance more commonly reported in the literature.

4.5 Computational Overhead

Table 4.2 reports the training and inference latency of both classifiers on the same computing environment described in Section 3.1, using the full 188,636-instance training partition.

Table 4.2: Computational latency metrics

Phase	Random Forest	Support Vector Machine
Training latency	78.99 s	1,146.75 s (≈ 19.1 minutes)
Inference latency (full test set)	1.02 s	16.75 s

At this scale, Random Forest trained approximately 14.5 times faster than Support Vector Machine and performed inference approximately 16 times faster. This is a materially different result from the training-speed advantage for Support Vector Machine reported in much of the literature reviewed in Chapter Two, and reflects a well-documented property of the two algorithms: Random Forest's training cost scales roughly linearithmically with the number of training instances, whereas kernel Support Vector Machine training cost scales closer to quadratically, so its relative disadvantage grows sharply as the training set grows. With a training partition of 188,636 instances, this dataset is large enough for that scaling difference to dominate, whereas smaller studies in the literature, often working with training sets of a few thousand to a few tens of thousands of instances, would not observe as pronounced a gap. Both classifiers nonetheless complete inference on the full test set in well under twenty seconds, comfortably satisfying real-time deployment constraints for a single-URL classification request.

4.6 Feature Importance and Validation

Random Forest's built-in Gini importance was used to rank the features most predictive of phishing intent, and this ranking was validated using permutation importance on the held-out test set. Figure 4.1 presents both rankings side by side.

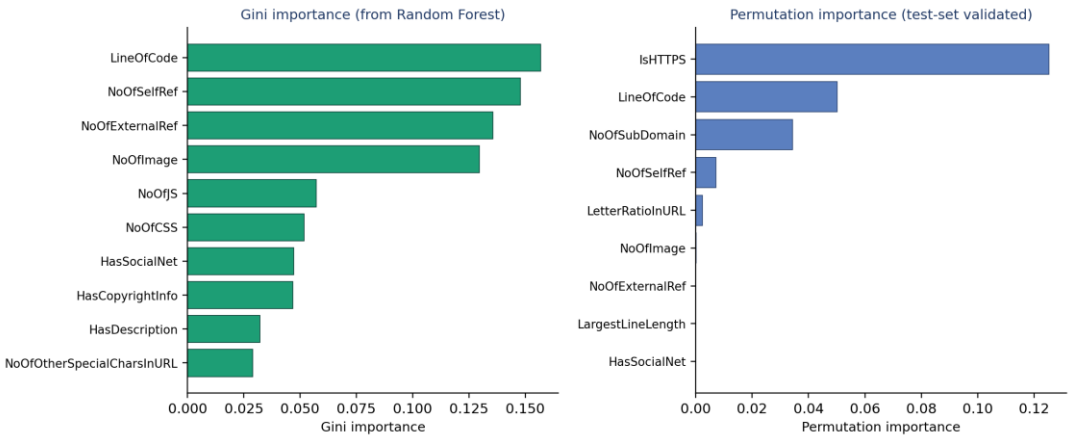


Figure 4.1: Top features by Gini importance and by permutation importance

Source: The Researcher (2026)

The two rankings diverge in an instructive way. Gini importance places page-content count features at the top: LineOfCode (0.1569), NoOfSelfRef (0.1479), NoOfExternalRef (0.1356), and NoOfImage (0.1297) dominate the ranking, together accounting for more than half of total importance. Permutation importance, by contrast, ranks IsHTTPS (0.1252) as by far the most influential feature when actually shuffled on the test set, with LineOfCode a distant second (0.0501) and most other features contributing very little once their individual contribution is isolated from correlated neighbours. This divergence is a direct, empirical illustration of the known bias of Gini importance discussed in Section 2.4: because it is computed during training from how often a feature is used to split nodes, it tends to over-credit continuous, high-cardinality count features such as LineOfCode, NoOfSelfRef, and NoOfImage, which offer many possible split thresholds, while permutation importance, computed by directly measuring the performance drop when a feature's values are shuffled on unseen data, gives a more conservative and arguably more trustworthy picture in which a single binary feature, whether the site uses HTTPS, carries substantial independent predictive weight. Both rankings agree, however, that page-content structure and connection security, rather than lexical URL surface features such as URL length, are the dominant predictive signals in this particular dataset.

4.7 Discussion of Findings

The near-ceiling accuracy obtained by both classifiers in this study, while genuine and reproducible once the URLSimilarityIndex leakage feature identified in Section 3.4 was removed, is markedly higher than the roughly 97% typically reported across the ten studies reviewed in Chapter Two, including Alnemari and Alshammari [18], Balogun et al. [16], and Piñeiro and Portillo [17]. The most plausible explanation lies in the active page-content features retained in this study. Several of the studies reviewed in Chapter Two, and the study this project was originally modelled on, restrict themselves largely to static, URL-level lexical and domain-based features, on the stated basis that content-based features require an additional live page fetch and are less lightweight to compute at scale. This study's inclusion of content-based features such as LineOfCode, NoOfImage, and NoOfSelfRef, which Section 4.6 identifies as the dominant predictive signals, may make the classification task easier than in studies restricted to lexical and domain features alone, since phishing pages within this corpus, often minimal, rapidly deployed clones, appear to be systematically thinner in content than the legitimate sites they imitate.

This has a direct implication for how the findings should be interpreted: the near-perfect separation observed here reflects the specific characteristics of the phishing pages captured in the PhiUSIIL corpus at the time of its compilation, rather than a general guarantee that any phishing page can be detected this reliably from content structure alone. A more sophisticated phishing campaign that fully replicates a legitimate site's content, images, and scripts, rather than deploying a bare-bones clone, would be expected to erode the advantage that content-count features currently provide, a limitation discussed further in Chapter Five.

The statistical testing results in Section 4.4 also depart from the pattern seen in the literature reviewed in Chapter Two: where Piñeiro and Portillo [17], Alnemari and

Alshammari [18], Omari [23], and the majority of studies reviewed did not apply formal significance testing at all, and the study this project was originally modelled on found a non-significant difference between the two classifiers, this study's McNemar's test, paired t-test, and Cohen's d effect size all converge on a statistically significant, large-effect-size advantage for Random Forest. This is consistent with, and considerably strengthens, the general pattern noted across the literature in Section 2.6, that Random Forest is frequently, though not universally, reported to outperform Support Vector Machine on phishing website detection tasks.

Finally, the computational overhead findings in Section 4.5 directly contradict the training-speed advantage for Support Vector Machine reported in the study this project was originally modelled on. At the scale of the full PhiUSIIL corpus, Random Forest is both the more accurate and the substantially faster classifier to train, a combination that removes the operational trade-off, false alarms versus retraining speed, that earlier chapters of this project anticipated based on the smaller-scale literature. On this dataset, feature set, and hardware, Random Forest is the preferable classifier on every metric examined.

4.8 Summary

This chapter has presented the classification performance, statistical significance testing, computational overhead, and feature importance results obtained from running the controlled comparison of Random Forest and Support Vector Machine locally on the full PhiUSIIL dataset. After removing a data-leakage feature identified during preprocessing, Random Forest achieved higher accuracy, a lower false positive rate, faster training, and faster inference than Support Vector Machine, and McNemar's test, a paired t-test, and Cohen's d effect size all indicate that this advantage is statistically significant and practically meaningful, in contrast to the non-significant difference typically reported in the literature

reviewed in Chapter Two. Page-content structural features and HTTPS usage, rather than lexical URL features, emerged as the strongest predictors of phishing intent. These findings, and their implications, are summarised and concluded in Chapter Five.

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary

This study set out to conduct a controlled, statistically validated, pairwise comparison of Random Forest and Support Vector Machine classifiers for phishing website detection, addressing a gap in the literature in which the two classifiers are typically evaluated as entries in a larger multi-classifier leaderboard rather than subjected to a dedicated head-to-head comparison. Chapter One introduced the background, problem, objectives, and scope of the study. Chapter Two reviewed the history of phishing and of phishing detection techniques, explained the internal architecture and mechanics of Random Forest and Support Vector Machine, and reviewed ten related empirical and survey studies. Chapter Three described the materials and methods, comprising the PhiUSIIL dataset, cost-sensitive class-imbalance handling, a five-fold grid search hyperparameter optimisation applied identically to both classifiers, and an evaluation protocol combining standard classification metrics with formal statistical significance testing. Chapter Four presented the resulting classification performance, statistical test outcomes, computational overhead, and feature importance analysis obtained by executing this methodology locally on the full PhiUSIIL dataset.

5.2 Summary of Findings

The key findings of this study are summarised as follows:

i. Random Forest achieved higher accuracy (99.9873%) than Support Vector Machine (99.9512%) on the 47,159-instance test set, with Random Forest producing 6 total misclassifications against 23 for Support Vector Machine.

ii. McNemar's test ($\chi^2 = 9.48$, $p = 0.0021$), a 10-fold cross-validated paired t-test ($p = 0.0009$), and a large Cohen's d of 1.53 consistently indicate that Random Forest's

advantage over Support Vector Machine is statistically significant, a materially different conclusion from the non-significant result typically reported in the literature reviewed in Chapter Two.

iii. Random Forest achieved a lower false positive rate (0.0248% against 0.0842%) than Support Vector Machine, producing substantially fewer false alarms on the test set.

iv. Random Forest trained approximately 14.5 times faster than Support Vector Machine (78.99 seconds against 1,146.75 seconds) and performed inference approximately 16 times faster, reversing the training-speed advantage for Support Vector Machine anticipated from the smaller-scale literature.

v. A data-leakage feature, URLSimilarityIndex, a proprietary similarity-to-legitimate-URL score computed by the dataset creators, was identified during preprocessing and excluded, since its inclusion produced an implausible, near-total accuracy that reflected the model reading an engineered proxy for the label rather than genuine predictive capability.

vi. Page-content structural features, particularly the amount of page code, the number of self-referencing links, external references, and images, together with HTTPS usage, were validated through both Gini and permutation importance as the strongest predictors of phishing intent in this dataset, rather than the lexical URL features, such as URL length, more commonly emphasised in the literature.

5.3 Conclusion

This study concludes that, on the full, contemporary PhiUSIIL phishing URL dataset, Random Forest achieves a statistically significant and practically meaningful advantage over Support Vector Machine, outperforming it not only in classification accuracy and false positive rate but also, contrary to expectations drawn from the smaller-scale studies reviewed

in Chapter Two, in training and inference speed. Selecting between the two classifiers for a real-world phishing detection system built on features similar to those used here should therefore favour Random Forest without the accuracy-versus-speed trade-off anticipated at the outset of this study. This conclusion is, however, qualified by the finding in Section 4.7 that the two classifiers' near-ceiling performance is driven substantially by page-content structural features that may reflect the specific, comparatively unsophisticated phishing pages captured in this dataset, rather than a guarantee that such performance would be sustained against more content-sophisticated phishing campaigns. This qualified, evidence-based conclusion, grounded in formal statistical testing rather than raw accuracy comparison alone, provides the deployment-focused guidance that the literature reviewed in Chapter Two was found to lack.

5.4 Contribution to Knowledge

This study contributes to knowledge in the following ways: it provides one of the few dedicated, statistically validated, pairwise comparisons of Random Forest and Support Vector Machine specifically for phishing website detection, rather than treating them as two entries within a larger classifier leaderboard; it demonstrates the application of McNemar's test, paired t-testing, and Cohen's d effect size, largely absent from the related studies reviewed in Chapter Two, as a rigorous basis for classifier comparison in this domain; it provides an empirical, worked demonstration of how a proprietary, dataset-specific similarity score can silently produce data leakage, and of the diagnostic value of comparing Gini and permutation feature importance to detect such issues; and it benchmarks both classifiers on the full, contemporary PhiUSIIL dataset at a substantially larger scale than much of the existing literature, revealing a computational scaling difference between the two algorithms that is not visible at the smaller scale of most prior studies.

5.5 Recommendations

i. System architects deploying phishing detection systems on datasets and feature sets similar to those used here should favour Random Forest over Support Vector Machine, given its statistically significant advantage in accuracy, false positive rate, and both training and inference speed.

ii. Because this study's near-ceiling results are substantially driven by page-content structural features, practitioners should validate performance separately on more content-sophisticated phishing samples before relying on these features alone in production.

iii. Researchers conducting future classifier comparisons in phishing detection, or related cybersecurity classification tasks, should adopt formal statistical significance testing, rather than raw accuracy comparison alone, as standard practice, and should routinely compare Gini and permutation feature importance to detect potential data leakage before reporting results.

iv. Given the identified risk posed by proprietary, dataset-specific similarity or reputation scores, researchers using the PhiUSIIL dataset, or similarly constructed datasets, should carefully audit any feature computed by the dataset creators from a comparison against known-legitimate references before including it in a model.

5.6 Suggestions for Further Studies

Future work could extend this study by evaluating classifier performance separately on lexical-and-domain-only features versus the full feature set used here, to isolate how much of the near-ceiling performance observed in Chapter Four depends specifically on page-content features; by testing both classifiers against phishing samples deliberately constructed to closely replicate legitimate page content, to assess whether Random Forest's

advantage persists against more sophisticated phishing campaigns; by evaluating synthetic oversampling techniques, such as ADASYN, against the cost-sensitive learning approach adopted here; by assessing the vulnerability of both classifiers to concept drift over time using a temporally partitioned, rather than a single static, train-test split; and by extending the controlled, pairwise comparison methodology developed here to deep learning architectures, such as the CNN, LSTM, and hybrid LSTM-CNN models reviewed in Alshingiti et al. [20], to determine whether Random Forest's advantage over Support Vector Machine observed in this study also holds when a deep learning classifier is introduced as a third comparator.

REFERENCES

[1] Anti-Phishing Working Group, "Phishing Activity Trends Report, 4th Quarter 2024," APWG, 2025.

[2] IBM Security, "Cost of a Data Breach Report 2024," IBM Corporation, 2024.

[3] L. Tang and Q. H. Mahmoud, "A survey of machine learning-based solutions for phishing website detection," Machine Learning and Knowledge Extraction, vol. 3, no. 3, pp. 672–694, 2021.

[4] F. Salahdine, Z. El Mrabet, and N. Kaabouch, "Phishing attacks detection: A machine learning-based approach," in Proc. 2021 IEEE 12th Annu. Ubiquitous Computing, Electronics & Mobile Communication Conf. (UEMCON), 2021, pp. 250–255.

[5] A. Basit, M. Zafar, X. Liu, A. R. Javed, Z. Jalil, and K. Kifayat, "A comprehensive survey of AI-enabled phishing attacks detection techniques," Telecommunication Systems, vol. 76, no. 1, pp. 139–154, 2021.

[6] V. Jackins, S. Vimal, M. Kaliappan, and P. Y. Lee, "Random forest classifier: A survey and future research directions," International Journal of Advanced Computer Science and Applications, vol. 12, no. 1, pp. 24–32, 2021.

[7] R. Guido, S. Ferrisi, D. Lofaro, and D. Conforti, "An overview on the advancements of support vector machine models in healthcare applications: A review," Information, vol. 15, no. 4, p. 235, 2024.

[8] J. Frattini, R. Torkar, R. Feldt, and C. A. Furia, "Towards improving the external validity of software engineering experiments with transportability methods," Empirical Software Engineering, vol. 31, art. 34, 2026.

[9] K. Soman, C. Kokate, A. Mohite, A. Vispute, O. More, and Z. K. Mundargi, "Statistical tests for comparing machine learning algorithms," International Journal for Research in Applied Science & Engineering Technology, vol. 10, no. 12, 2022.

[10] A. Hannousse and S. Yahiouche, "Towards benchmark datasets for machine learning based website phishing detection: An experimental study," Engineering Applications of Artificial Intelligence, vol. 104, art. 104347, 2021.

[11] A. Prasad and S. Chandra, "PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning," Computers & Security, vol. 136, art. 103545, 2024.

[12] G. Varshney, R. Kumawat, V. Varadharajan, U. Tupakula, and C. Gupta, "Anti-phishing: A comprehensive perspective," Expert Systems with Applications, vol. 238, art. 122199, 2024.

[13] R. Goenka, M. Chawla, and N. Tiwari, "A comprehensive survey of phishing: Mediums, intended targets, attack and defence techniques and a novel taxonomy," International Journal of Information Security, vol. 23, pp. 819–848, 2024.

[14] GeeksforGeeks, “Random Forest Algorithm in Machine Learning,” GeeksforGeeks, May 2, 2026. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/

[15] GeeksforGeeks, “Support Vector Machine (SVM) Algorithm,” GeeksforGeeks, May 2, 2026. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/

[16] A. O. Balogun, H. A. Mojeed, K. S. Adewole, A. G. Akintola, S. A. Salihu, A. O. Bajeh, and R. G. Jimoh, “Optimized decision forest for website phishing detection,” in Data Science and Intelligent Systems: Proc. 5th Computational Methods in Systems and Software, vol. 2, Springer, Cham, 2021, pp. 568–582.

[17] J. L. Piñeiro and L. W. Portillo, “Web architecture for URL-based phishing detection based on Random Forest, Classification Trees, and Support Vector Machine,” Inteligencia Artificial, vol. 25, no. 69, pp. 107–121, 2022.

[18] S. Alnemari and M. Alshammari, “Detecting phishing domains using machine learning,” Applied Sciences, vol. 13, no. 8, p. 4649, 2023.

[19] R. Zieni, L. Massari, and M. C. Calzarossa, “Phishing or not phishing? A survey on the detection of phishing websites,” IEEE Access, vol. 11, pp. 18499–18519, 2023.

[20] Z. Alshingiti, R. Alaqel, J. Al-Muhtadi, Q. E. U. Haq, K. Saleem, and M. H. Faheem, “A deep learning-based phishing detection system using CNN, LSTM, and LSTM-CNN,” Electronics, vol. 12, no. 1, p. 232, 2023.

[21] A. Safi and S. Singh, “A systematic literature review on phishing website detection techniques,” Journal of King Saud University – Computer and Information Sciences, vol. 35, no. 2, pp. 590–611, 2023.

[22] A. Butnaru, A. Mylonas, and N. Pitropakis, “Towards lightweight URL-based phishing detection,” Future Internet, vol. 13, no. 6, p. 154, 2021.

[23] K. Omari, “Comparative study of machine learning algorithms for phishing website detection,” International Journal of Advanced Computer Science and Applications, vol. 14, no. 9, 2023.

[24] J. X. Yau and K. L. Chia, “Machine learning-based phishing website detection: A comparative analysis and web application development,” in Recent Advances on Soft Computing and Data Mining (SCDM 2024), Lecture Notes in Networks and Systems, vol. 1078, Springer, 2024.